

Programmazione 1

Corso c

>Correttezza di una
sequenza di assegnazioni

Alessandro Mazzei

Dipartimento di Informatica

Università degli Studi di Torino



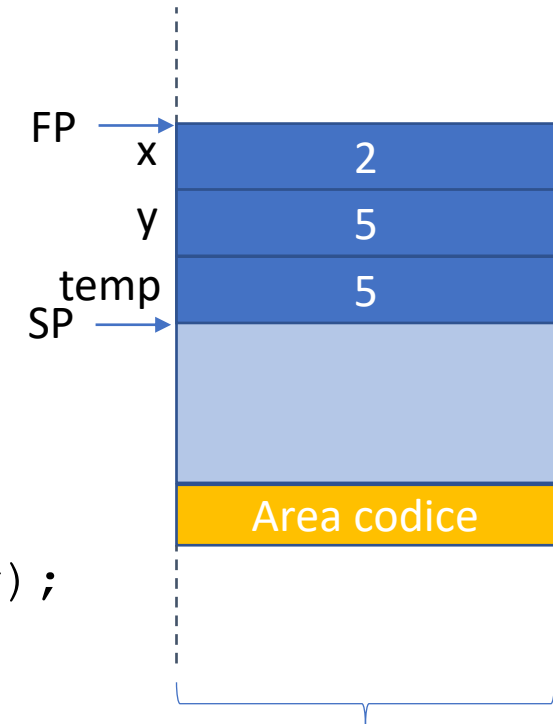
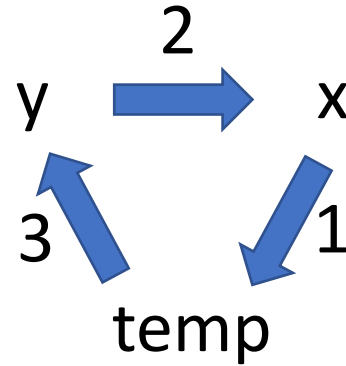
UNIVERSITÀ
DI TORINO

Correttezza

- Fino ad ora abbiamo visto degli algoritmi molto semplici basati su *sequenze di assegnazioni*, come lo **swap** di due valori con un buffer
- Essendo semplici, *ad occhio*, siamo certi che questi algoritmi sono **corretti** ovvero che producono sempre un output corretto su qualsiasi input

SWAP

```
int main(void) {  
    int x = 5;  
    int y = 2;  
    int temp;  
  
    temp = x;  
    x = y;  
    y = temp;  
    printf ("Le var x, y e valgono %d, %d\n", x, y);  
}
```



```
$ ./swap_2  
'Le var x, y e valgono 2, 5
```

Correttezza

- Nelle prossime lezioni le cose si complicheranno, quando introdurremo anche i concetti (e i costrutti) di *scelta* e *ripetizione*, necessari, insieme alla *sequenza*, per implementare un qualsiasi algoritmo (teorema di Bohm-Jacopini)
- Quando i programmi sono complessi è difficile (impossibile!) essere certi ad occhio della correttezza di un algoritmo

Correttezza formale

- Si parla di correttezza formale quando riusciamo a **dimostrare** che un algoritmo è corretto per ogni input **possibile** (ogni input che *garantisce* certe specifiche proprietà).
- Noi lo vedremo nel caso più semplice ovvero con algoritmi che usano solo assegnazioni, ma in linea di principio è possibile ampliare il formalismo a qualsiasi algoritmo
- Quindi, in questa lezione, algoritmo o programma sono sinonimi di *sequenza di assegnazioni*

Obiettivo: SWAP2

- Vogliamo dimostrare che questo algoritmo:

```
{  
  int a = 1;  
  int b = 3;  
  
  a = a + b;  
  b = a - b;  
  a = a - b;  
} //a=? b=?
```

Inverte correttamente i valori iniziali delle variabili intere a e b.

N.B niente terza variabile!

<assert.h>

- Intruciamo la macro **assert**, ovvero una funzione (che lavora in *preprocessing*) che controlla la verità di una condizione booleana:
 - Se **true**, va avanti senza far nulla
 - Se **false**, interrompe un messaggio, restituendo delle informazioni

```
{  
    int a = 1;  
    int b = 3;  
    assert (a+b!=4) ;  
}
```

```
$ ./a.out2  
Assertion failed: (a+b!=4), function main, file test.c, line 8.  
Abort trap: 6
```

- no debugging? -> NDEBUG

#define

- Introduciamo **#define**, ovvero un'istruzione che ci consente di definire le costanti in pre-processing

```
# define X 3
{
    int a = X;
    assert (a==3) ;
}
```


Outline

- Stati di un programma e predicati
- Pre-condizione e Post-condizione
- Domande sulla correttezza
- Logica di Floyd
 - Backward reasoning
 - Weakest Precondition
- SWAP2
- Esempi di esercizi d'esame

Outline

- Stati di un programma e predicati
- Pre-condizione e Post-condizione
- Domande sulla correttezza
- Logica di Floyd
 - Backward reasoning
 - Weakest Precondition
- SWAP2
- Esempi di esercizi d'esame

Semantica assiomatica: stato di un programma

- Introduciamo il concetto di **stato di un programma**, che consiste nel verificare l'uguaglianza dei **valori delle variabili** in un certo punto nell'esecuzione del programma
- Per farlo useremo delle **espressioni booleane** (atomiche o complesse) sui valori assunti dalle variabili
- In logica si chiamano **predicati** (*logica dei predicati*)

Esempio: stati 1, 2 e 3

```
{  
  int a = 3;  
  // (1) stato descritto col predicato a==3  
  int b = 12;  
  // (2) stato descritto col predicato a==3 && b==12  
  int y = a + b;  
  // (3) stato descritto col predicato a==3 && b==12 && y==15  
}
```

Esempio: stati 1, 2 e 3

```
{  
    int a = 3;  
    // (1) stato descritto col predicato a==3  
    assert(a == 3);  
    int b = 12;  
    // (2) stato descritto col predicato a==3 && b==12  
    assert(a == 3 && b == 12);  
    int y = a + b;  
    // (3) stato descritto col predicato a==3 && b==12 && y==15  
    assert((a == 3 && b == 12) && (y == 15));  
}
```

Logica booleana

- Stiamo usando la logica booleana (operatori **&&**, **||**, **!**) sulle uguaglianze, ovvero delle espressioni complesse che possono essere vere o false sulla base della verità delle singole uguaglianze
- **Tavole di verità:**

E'	E''	E = E' && E''
0	0	0
0	1	0
1	0	0
1	1	1

E'	E''	E = E' E''
0	0	0
0	1	1
1	0	1
1	1	1

E'	E
0	1
1	0

Logica booleana

• Stiamo usando la logica booleana (operatori &&, ||, !) sulle uguaglianze, ovvero delle espressioni complesse che possono essere vere o false sulla base della verità delle singole uguaglianze

IMPORTANTE: la logica booleana gioca un ruolo fondamentale nei costrutti di scelta e ripetizione!!!

- **Tavole di verità:**

E'	E''	E = E' && E''
0	0	0
0	1	0
1	0	0
1	1	1

E'	E''	E = E' E''
0	0	0
0	1	1
1	0	1
1	1	1

E'	E
0	1
1	0

Stato Iniziale e Stato Finale

- Lo **Stato Iniziale** è descritto tramite un predicato che fissa valori e relazioni dell'insieme di variabili **all'inizio** di un qualsiasi algoritmo
- Lo **Stato Finale** è descritto tramite un predicato che fissa valori e relazioni dell'insieme di variabili **al termine** di un qualsiasi algoritmo

```
// (0) stato iniziale
{
  // (1)
  a = x + 3;
  // (2)
  y = a + b;
}
// (3) stato finale
```


Outline

- Stati di un programma e predicati
- Pre-condizione e Post-condizione
- Domande sulla correttezza
- Logica di Floyd
 - Backward reasoning
 - Weakest Precondition
- SWAP2
- Esempi di esercizi d'esame

Pre-condizione e post-condizione di un'istruzione

Si definisce di **Pre-condizione** (**Post-condizione**) di un'istruzione il predicato che si trova **prima** (**dopo**) l'istruzione.

```
{  
  // (1) P1  
  a = x + 3; // I1  
  // (2) P2  
  y = a + b; // I2  
  // (3) P3  
}
```

- P1 è pre-condizione di I1
- P2 è pre-condizione di I2 e post-condizione di I1
- P3 è post-condizione di I2

Pre-condizione del programma

- Una **pre-condizione del programma** è un predicato che si può assumere essere vero all'inizio del programma

```
// (PRE) Pre-condizione: 10 <= x && (b == 32 - x || b == 16)
// (0) stato iniziale
{
  // (1)
  a = x + 3;
  // (2)
  y = a + b;
  // (3) stato finale
}
```

- N.B. prima di // (0)

Post-condizione del programma

- Una **post-condizione del programma** è un predicato che si può assumere essere vero alla fine del programma

```
// (PRE) Pre-condizione: 10 <= x && (b == 32 - x || b == 16)
// (0) stato iniziale
{
  // (1)
  a = x + 3;
  // (2)
  y = a + b;
  // (3) stato finale
}
// (POST) Post-condizione: 25 <= y
```

- N.B. dopo di // (3)

Outline

- Stati di un programma e predicati
- Pre-condizione e Post-condizione
- Domande sulla correttezza
- Logica di Floyd
 - Backward reasoning
 - Weakest Precondition
- SWAP2
- Esempi di esercizi d'esame

Predicati su stati: perché?

- I predicati logici ci permettono di **dimostrare** una **tesi** a partire da una **ipotesi** mediante *inferenza logica*, ovvero una forma di calcolo che si basa sulle tavole di verità degli operatori booleani e sull'implicazione →
- Quindi possiamo partire dalla verità di un predicato P1 su uno stato (**ipotesi**) per verificare la verità di un predicato P2 un altro stato (**tesi**).
- Usando la logica, possiamo quindi rispondere a diversi tipi di domande sugli stati di un programma

Alcune domande tipiche sulla correttezza che posso risolvere con la LdF

1. Pre $\rightarrow$ Post?

2. Supponiamo che in `//(3)` valga il predicato $a < b$, quale deve essere il predicato più generale possibile in `// (0)` affinché ciò avvenga?

3. Supponiamo che in `//(1)` valga il predicato $12 \leq x \ \&\& \ a \% 2 == 0$, quali stati possiamo raggiungere al punto (2)?

```
// (PRE)
// (0) stato iniziale
{
    // (1)
    a = x + 3;
    // (2)
    y = a + b;
}
// (3) stato finale
// (POST)
```

Alcune domande tipiche sulla correttezza che posso risolvere con la LdF

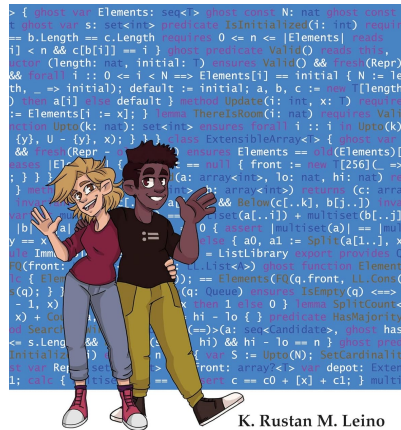
1. Pre $\rightarrow$ Post?

2. Supponiamo che in `//(3)` valga il predicato $a < b$, quale deve essere il predicato più generale possibile in `// (0)` affinché ciò avvenga?
3. Supponiamo che in `//(1)` valga il predicato $12 \leq x \ \&\& \ a \% 2 == 0$, quali stati possiamo raggiungere al punto (2)?

```
// (PRE)
// (0) stato iniziale
{
    // (1)
    a = x + 3;
    // (2)
    y = a + b;
}
// (3) stato finale
// (POST)
```


Riferimenti bibliografici

- Capitolo 5 Indice_Lezioni del prof. Roversi
- https://it.wikipedia.org/wiki/Logica_di_Hoare
 - Sezioni: Tripla di Hoare, Assioma dell'assegnamento, Regola della conseguenza.
- Cap. 2 di Leino&Leino. **Program Proofs**. MIT Press, 2023.



K. Rustan M. Leino
illustrated by Kaleb Leino

PROGRAM PROOFS

